Literature Review
P2P File Synchronization System

Action Learning Project

Backend (Go, C++)

Arihant Jain
Su Huizhe

Frontend and CI/CD (Java, GitHub Actions)

Nizar Soufiya
Shengkang Duan

May 28, 2026

Contents

1 Problem Context

TODO (team): Problem Context Section

2 Overview of Existing Work

TODO (team): Overview of Existing Work Section

3 Comparison of Approaches

To establish the context for our proposed peer-to-peer (P2P) file synchronization system, we compare the primary collaborative sharing paradigms: cloud-based directory services, distributed version control systems, and decentralized peer-to-peer file synchronization.

3.1 Evaluation of Existing Paradigms

1. **Cloud Directory Services** (e.g., Google Drive, Dropbox, Nextcloud):
   These tools rely on centralized cloud servers. While they provide easy setup and work well for simple documents, they suffer from high latency, transfer size limits, and reliance on external internet connections. Furthermore, centralized storage raises data privacy concerns, and administrative access control rules often create collaboration bottlenecks.

2. **Version Control Systems** (e.g., Git, SVN):
   VCS tools track history using directed acyclic graphs (DAGs). Although they offer powerful conflict resolution and change logs, they are designed for source code. They struggle with large binary files and require manual execution of staging, committing, pushing, and pulling. This workflow introduces steep learning curves for non-technical users.

3. **P2P File Synchronization** (e.g., Syncthing, Resilio Sync):
   Decentralized sync programs replicate folder states directly between local devices. They eliminate cloud dependencies and storage quotas, providing fast LAN transfers. However, they lack integrated version rollback histories, leaving systems vulnerable to accidental overwrites. Additionally, setup requires manual peer ID and folder sharing approvals, adding configuration friction for temporary project groups.

3.2 Key Product Differences: Syncthing vs. Resilio Sync

We highlight the differences between the two leading peer-to-peer sync tools to clarify our design choices:

- **Architecture and Openness:** Syncthing is open-source and uses the public Block Exchange Protocol (BEP). It enforces a strict security model where devices must exchange long cryptographic IDs and mutually approve folder invitations. Resilio Sync is proprietary, uses the BitTorrent protocol, and offers selective sync, but has commercial license limits and proprietary lock-in.

- **Lightweight Collaboration Gap:** Both tools serve as generic directory mirrors rather than project-specific workspaces. They lack automated, non-destructive version histories that team members can navigate easily. If a peer saves a broken draft, that file immediately overwrites the state of all other online peers, requiring complex administrative configuration to recover.

3.3 Design Goals of Our Project

Our system aims to bridge the gap between the speed of P2P file sync and the safety of version control:

- **Zero Configuration (planned):** Use Multicast DNS (mDNS) [3] and DNS-SD [2] to discover local peers without manual setup.
- **Local Transfers (planned):** Use direct TCP socket connections [9] to exchange data over the local network and avoid cloud roundtrips.
- **Peer Autonomy (planned):** Run without a central server so peers exchange updates directly.
- **Durability (planned):** Use a Last-Write-Wins (LWW) policy with local backups before overwrite.

3.4 Structured Comparison Table

Table 1 evaluates the feature tradeoffs of the three paradigms.

Table 1: Comparison of Collaborative File Sharing and Synchronization Approaches

Feature / Dimension	**Cloud Directory Services**	**Version Control Systems**	**P2P File Sync (Proposed)**
Primary Topology	Centralized Client-Server	Distributed (DAG-based)	Peer-to-Peer (Mesh)
Network Discovery	Centralized cloud portal	Remote repository server	Local mDNS / LAN broadcast
Setup Complexity	Account setup	Repository creation / auth	Low (auto-discovery planned)
Target Use Case	Generic office documents	Source code tracking	Live directory mirroring
Conflict Resolution	Duplicate file generation	Manual merges / diff resolution	LWW (planned)
Version Control	Periodic file revisions	Explicit commits and branches	Local backups (planned)
File Watcher	Periodic polling	Manual staging	Kernel events (planned)
Bandwidth Efficiency	Server upload/download	Incremental compression diffs	Direct sockets / planned delta sync
Internet Dependency	High (cloud servers required)	High (for pull/push)	None (offline-capable)

Continued on next page

Table 1 – Continued from previous page

Feature / Dimension	Cloud Directory Services	Version Control Systems	P2P File Sync (Proposed)
Data Privacy	Low (third-party storage)	Medium (third-party servers)	High (local-only storage)
Access Permissions	Centralized ACL approvals	Branch rules and PR reviews	Peer-to-peer access

4 Identified Gap

TODO (team): Identified Gap Section

5 Positioning Your Project

TODO (team): Positioning Your Project Section

6 Research to Application Mapping

To translate academic theories into our proposed architecture, we map six core research insights to planned components in our Go/C++ design:

1. **Zero-Configuration Peer Discovery on LAN**
   - **Research Insight:** Cheshire and Krochmal [3, 2] establish that Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD) enable serverless local service registration and resolution using standard IP multicast queries.
   - **Application in Project:** We plan a Go-based peer discovery component that broadcasts service records under the `_p2psync._tcp` domain and browses the local subnet to establish direct peer connections without manual setup.
2. **Causal Ordering of Concurrent Updates**
   - **Research Insight:** Lamport [6] demonstrates that logical clocks and causal relation tracking can order events in a distributed system without relying on synchronized physical clocks.
   - **Application in Project:** We plan to maintain per-file logical timestamps (Lamport clocks) to order updates and detect concurrent edits. This metadata will let us trace revision history and flag conflicts.
3. **Consistency Strategies under Network Partitions**
   - **Research Insight:** Gilbert and Lynch's proof of the CAP Theorem [4] establishes that distributed systems must trade consistency for availability during network partitions.
   - **Application in Project:** We plan to allow offline edits. When connection is restored, peers resync, resolve conflicts, and converge on a common state.
4. **Conflict Handling and Durability**

- **Research Insight:** Replicated storage systems like Bayou [8] demonstrate that resolving conflicts requires deterministic policies paired with non-destructive version backups to ensure data durability.
- **Application in Project:** We plan to use a Last-Write-Wins (LWW) policy based on logical timestamps. Before overwriting a local file, we plan to store a backup copy to protect against data loss.

5. **Conflict-Free Convergence Evolution**

   - **Research Insight:** Shapiro et al. [10] prove that Conflict-Free Replicated Data Types (CRDTs) allow concurrent updates to merge automatically and converge mathematically.
   - **Application in Project:** We plan to start with LWW for simplicity and design metadata so we can evolve toward state-based CRDTs in later iterations.

6. **Bandwidth-Efficient Synchronization**

   - **Research Insight:** Tridgell and Mackerras [11] show that rolling checksums allow remote directories to synchronize by transferring only modified blocks instead of entire files.
   - **Application in Project:** We plan to start with whole-file transfers for simplicity. In a later optimization phase, we will add block-level diffing to minimize local WLAN bandwidth use.

7 Key References

The following six publications serve as the primary theoretical foundation for our system's architecture and design choices.

1. **RFC 6762 & RFC 6763 (Stuart Cheshire and Marc Krochmal, 2013)** [3, 2]
   These specifications outline Multicast DNS and DNS-SD protocols, which allow network nodes to discover services without central DNS servers. We plan to use these standards for a zero-configuration discovery service so student devices on the same WLAN can form a sync group without manual IP entry.

2. **Leslie Lamport (1978) - "Time, Clocks, and the Ordering of Events in a Distributed System"** [6]
   This paper introduces logical clocks to track event ordering and causality in distributed environments. We plan to apply this concept by attaching logical timestamps to each file so our coordination logic can identify conflicts and order changes.

3. **Seth Gilbert and Nancy Lynch (2002) - "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services"** [4]
   This work mathematically proves the CAP Theorem. It guides our architectural decision to prioritize availability (allowing offline file modification) and eventual convergence over immediate consensus. This fits a student project environment where devices connect and disconnect frequently.

4. **Marc Shapiro et al. (2011) - "A Comprehensive Study of Convergent and Commutative Replicated Data Types"** [10]
   This research report defines CRDTs, providing the theory behind conflict-free state merging. We use this work as a roadmap to design metadata so we can upgrade from Last-Write-Wins to CRDT-based merging in future versions.

5. **Andrew Tridgell and Paul Mackerras (1996) - "The Rsync Algorithm"** [11]
   This technical report details rolling checksum diffs to achieve delta sync. It serves as our reference for planned optimization phases so we can update large assets by sending only modified blocks.

6. **Karin Petersen et al. (1997) - "Flexible Update Propagation for Weakly Consistent Replicated Storage"** [8]
   This paper explores conflict management policies and reconciliation in weakly connected replicated storage systems. It informs our planned conflict-handling design: combine an automated Last-Write-Wins strategy with local version backups to maintain durability and user control.

8 Conclusion

8.1 Literature Review Synthesis

Our analysis of distributed systems literature highlights that building a serverless, local directory sync tool requires combining mature local network standards with formal concurrency reasoning. mDNS and DNS-SD [3, 2] eliminate the need for central registry infrastructure. Logical clocks [6] resolve ordering ambiguity in decentralized meshes, while the CAP Theorem [4] demands that we trade strict consistency for high availability to support dynamic node presence. Finally, non-destructive resolution strategies (like LWW with backups [8]) provide a safe baseline for file durability, which can be upgraded to CRDTs [10] in the future.

8.2 Influence on Design

The literature shapes our proposed Go/C++ architecture:

- We plan to use mDNS/DNS-SD (via Go) to automate local network peer discovery and reduce setup friction.
- We plan to use logical timestamps and a Last-Write-Wins conflict policy to manage concurrent edits predictably.
- We plan to favor local availability, allowing users to work offline and sync differences upon reconnection.
- We plan a split where Go handles network coordination and C++ handles OS-level file watching (inotify [5], FSEvents [1]) and hashing (SHA-256 [7]).

8.3 Limitations and Future Work

While our current design provides a synchronization baseline, we recognize limitations in bandwidth usage and conflict handling. Whole-file transfers are simple to implement but inefficient for large assets. Our next steps are:

- Implement the Go-to-C++ socket handover pipeline to connect the network and storage layers.
- Implement the rsync-style rolling checksum algorithm [11] in C++ to enable delta transfers.
- Introduce state-based CRDT schemas [10] in Go to support automatic, conflict-free metadata merging.

References

[1] Apple Inc. File system events programming guide. `https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/FSEvents_ProgGuide/Introduction/Introduction.html`, 2021.

[2] Stuart Cheshire and Marc Krochmal. DNS-based service discovery. RFC 6763, RFC Editor, February 2013.

[3] Stuart Cheshire and Marc Krochmal. Multicast DNS. RFC 6762, RFC Editor, February 2013.

[4] Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.

[5] Michael Kerrisk. inotify(7) — Linux programmer's manual, 2021. `https://man7.org/linux/man-pages/man7/inotify.7.html`.

[6] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[7] National Institute of Standards and Technology. Secure hash standard (SHS). FIPS PUB 180-4, NIST, 2015.

[8] Karin Petersen, Mike J. Spreitzer, Douglas B. Terry, Marvin M. Theimer, and Alan J. Demers. Flexible update propagation for weakly consistent replicated storage. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, pages 288–300, 1997.

[9] Jon Postel. Transmission control protocol. RFC 793, RFC Editor, September 1981.

[10] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. Research Report RR-7687, INRIA, 2011.

[11] Andrew Tridgell and Paul Mackerras. The rsync algorithm. Technical Report TR-CS-96-05, Australian National University, 1996.